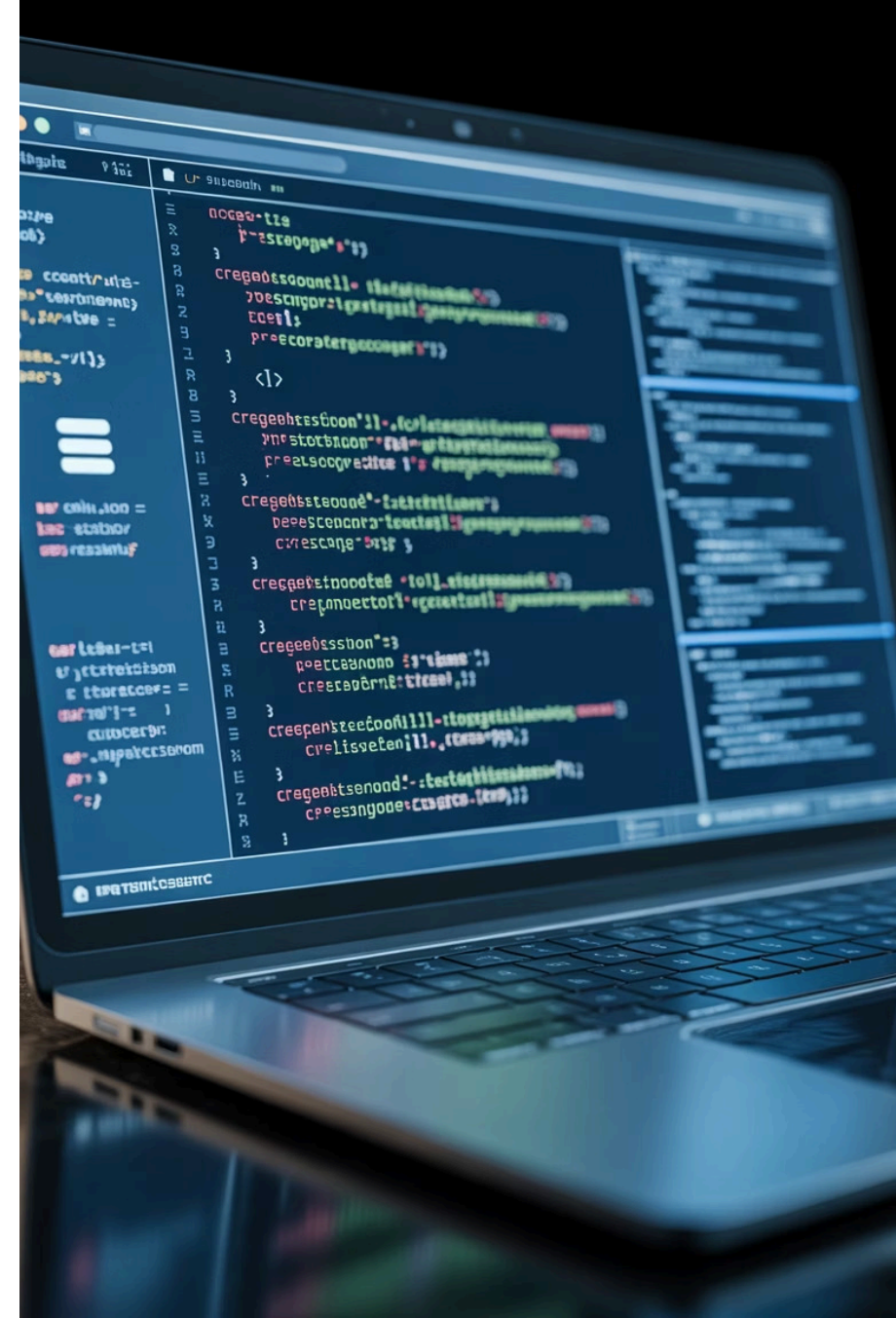


# Reusabilidade e Persistência de Dados

## Interfaces, Abstração e PDO

UNIVERSIDADE DO OESTE DE SANTA CATARINA – UNOESC  
Ciência da Computação – Programação II

Prof.: Leandro Otavio Cordova Vieira



# Objetivos da Aula



## Consolidar Reuso de Código

Aplicar princípios de reutilização para criar código mais eficiente e manutenível através de interfaces e abstração



## Aplicar Abstração e Interfaces

Dominar conceitos de orientação a objetos que garantem contratos e padronização entre classes



## Implementar Persistência com PDO

Utilizar PHP Data Objects para conexão segura e abstrata com bancos de dados relacionais



## Desenvolver CRUD Completo

Construir operações completas de Create, Read, Update e Delete aplicando todos os conceitos aprendidos



# Interfaces em PHP

## O que são Interfaces?

Interfaces funcionam como **contratos** entre classes, definindo um conjunto de métodos que devem ser obrigatoriamente implementados. Elas garantem um comportamento padronizado e previsível em toda a aplicação.

Uma interface não contém implementação, apenas assinaturas de métodos. Todas as classes que implementam uma interface devem fornecer a lógica concreta para cada método declarado.

### Características principais:

- Define métodos sem implementação
- Garante consistência entre classes
- Implementada usando a palavra-chave `implements`
- Uma classe pode implementar múltiplas interfaces



"Interfaces estabelecem o **que** deve ser feito, não **como** fazer."



# Exemplo de Interface: Repositorio

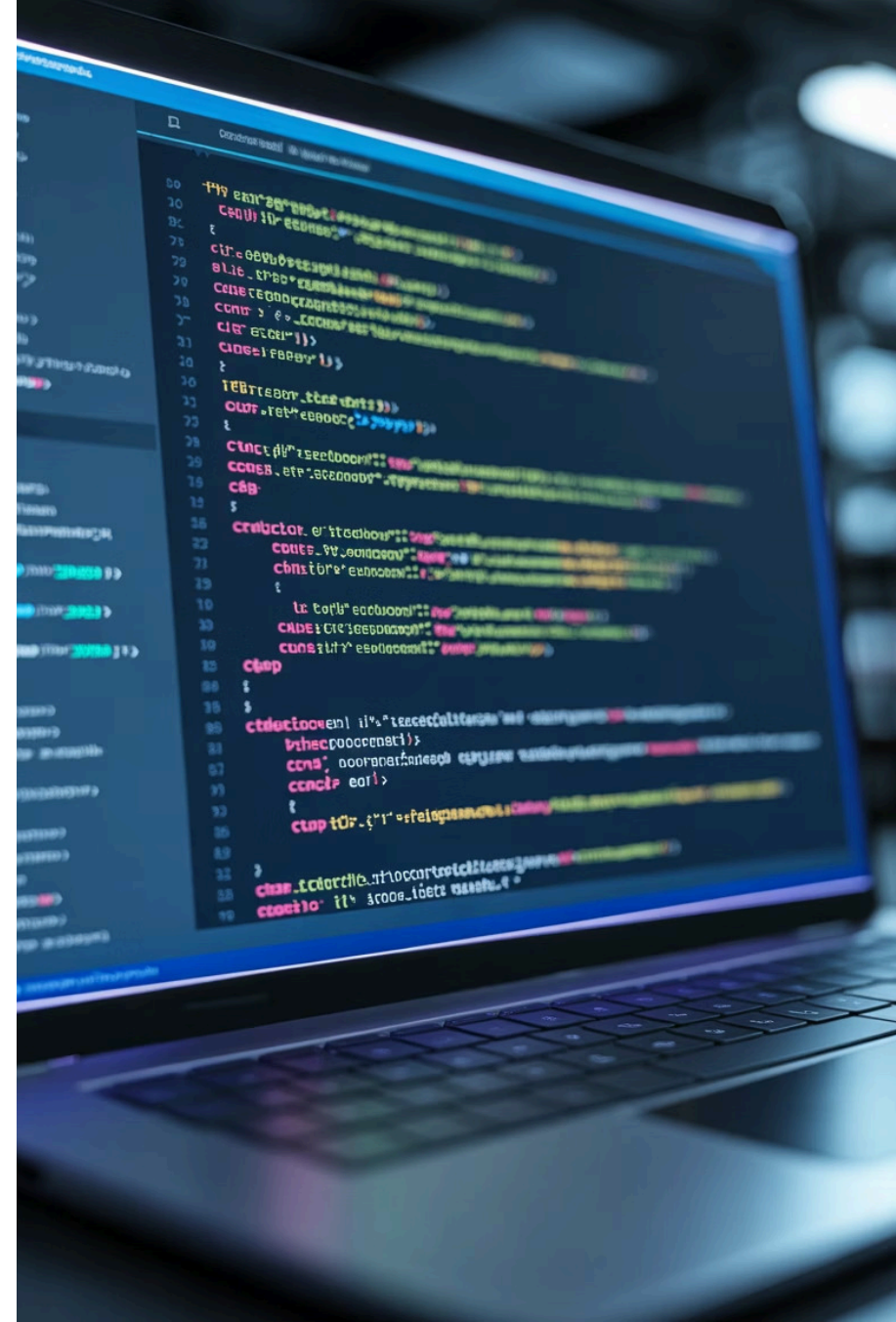
```
interface Repositorio {  
    public function salvar($obj);  
    public function listar();  
    public function buscarPorId($id);  
    public function atualizar($obj);  
    public function deletar($id);  
}
```

## Por que usar interfaces?

- Padroniza operações em diferentes repositórios
- Facilita manutenção e testes
- Permite trocar implementações facilmente

## Na prática

Qualquer classe que implemente Repositorio terá garantidamente esses métodos disponíveis, independente do banco de dados ou lógica interna utilizada.



# Classes Abstratas

01

## Modelo Base para Herança

Classes abstratas servem como **templates** para outras classes, fornecendo uma estrutura básica que deve ser estendida e completada pelas classes filhas.

02

## Métodos Abstratos e Concretos

Podem conter tanto métodos **abstratos** (sem implementação, que devem ser definidos nas classes filhas) quanto métodos **concretos** (com implementação completa e funcional).

03

## Implementação via abstract class

Declaradas usando a palavra-chave `abstract class` e estendidas pelas classes concretas através de `extends`.

- ❏ **Diferença chave:** Enquanto interfaces definem apenas *contratos*, classes abstratas podem fornecer *implementações parciais* que são herdadas pelas classes filhas.



# Exemplo de Classe Abstrata: Modelo

```
abstract class Modelo {  
    protected $id;  
    protected $dataCriacao;  
  
    // Método concreto - já implementado  
    public function getId() {  
        return $this->id;  
    }  
  
    public function setId($id) {  
        $this->id = $id;  
    }  
  
    // Método abstrato - deve ser implementado nas classes filhas  
    abstract public function validar();  
  
    // Método concreto com lógica comum  
    public function registrarCriacao() {  
        $this->dataCriacao = date('Y-m-d H:i:s');  
    }  
}
```

## Métodos Concretos

Métodos como `getId()` já têm implementação completa e podem ser usados diretamente pelas classes filhas.

## Métodos Abstratos

O método `validar()` deve ser obrigatoriamente implementado em cada classe que estender `Modelo`.

# PDO – PHP Data Objects

## Abstração de Banco de Dados

O PDO fornece uma **camada única de acesso** que funciona com diversos bancos de dados relacionais, incluindo MySQL, PostgreSQL, SQLite e Oracle. Isso permite trocar de banco sem reescrever todo o código.

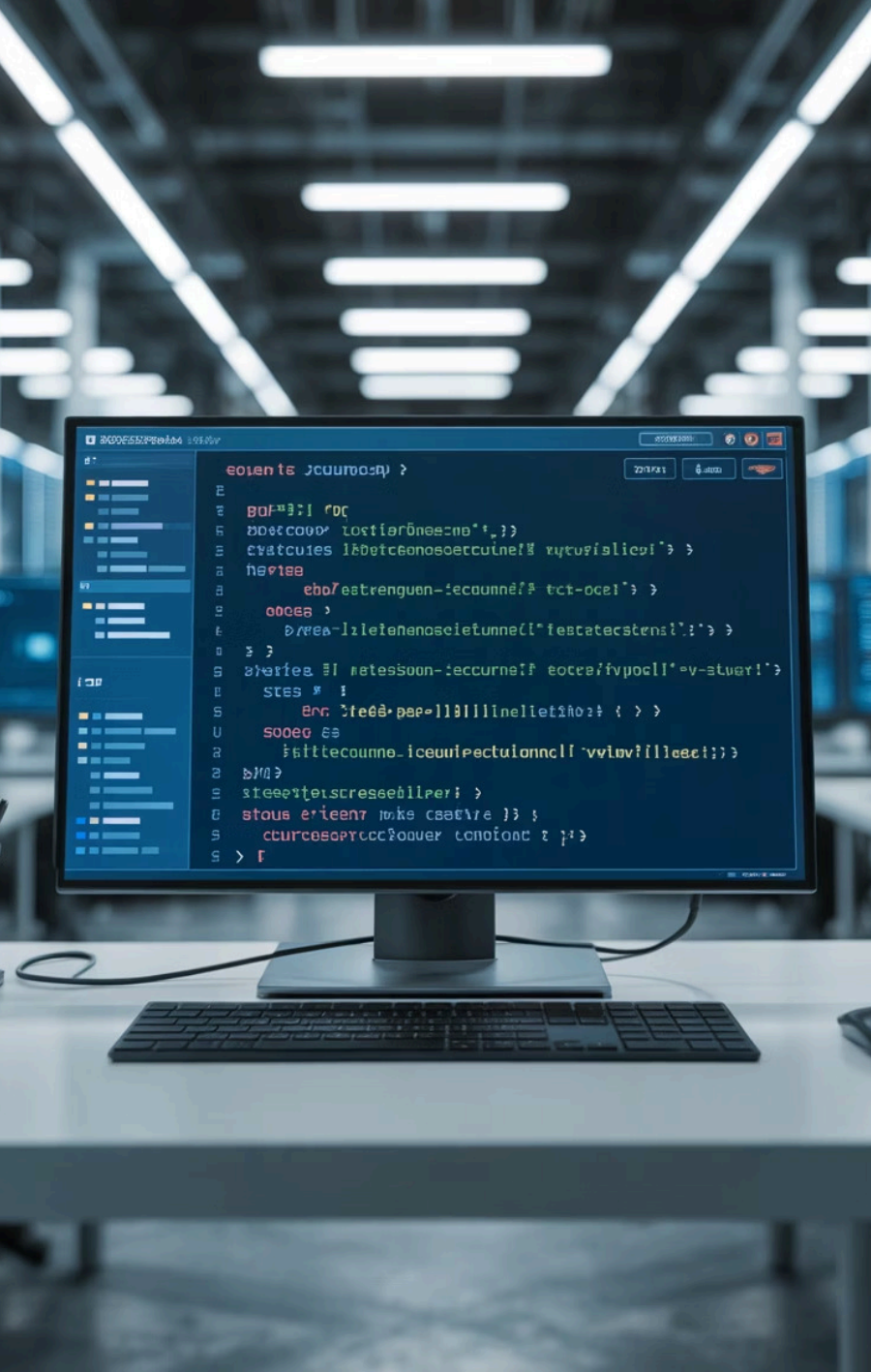
## Prepared Statements

Suporte nativo a `prepare()`, `bindParam()` e `execute()` permite criar consultas parametrizadas que separam código SQL de dados do usuário.

## Segurança Contra SQL Injection

Ao usar prepared statements, o PDO **evita automaticamente SQL Injection**, uma das vulnerabilidades mais perigosas em aplicações web. Os dados são tratados como valores, nunca como código executável.

PDO é a forma moderna e segura de conectar aplicações PHP a bancos de dados relacionais.





# Conectando com PDO

```
class Conexao {  
    private static $instancia = null;  
  
    public static function conectar() {  
        if (self::$instancia === null) {  
            try {  
                self::$instancia = new PDO(  
                    'mysql:host=localhost;dbname=escola;charset=utf8',  
                    'root',  
                    '',  
                    [  
                        PDO::ATTR_ERRMODE => PDO::ERRMODE_EXCEPTION,  
                        PDO::ATTR_DEFAULT_FETCH_MODE => PDO::FETCH_ASSOC,  
                        PDO::ATTR_EMULATE_PREPARES => false  
                    ]  
                );  
            } catch (PDOException $e) {  
                die("Erro na conexão: " . $e->getMessage());  
            }  
        }  
        return self::$instancia;  
    }  
}
```

- **Singleton Pattern**

Garante uma única instância de conexão, economizando recursos do servidor

- **Tratamento de Erros**

Configurado para lançar exceções em caso de problemas, facilitando debug

- **Modo de Busca**

Retorna resultados como arrays associativos por padrão





# Classe Concreta: Aluno

## Integrando Todos os Conceitos

A classe `Aluno` é um exemplo perfeito de como interfaces, classes abstratas e PDO trabalham juntos em harmonia:



### Implementa Interface

Cumpe o contrato definido por `Repositorio`, garantindo todos os métodos CRUD



### Herda Classe Abstrata

Estende `Modelo` para reaproveitar código comum e implementar métodos abstratos



### Usa PDO

Realiza operações de persistência de forma segura através do PDO



Esta arquitetura promove **separação de responsabilidades, reuso de código e facilidade de manutenção.**

# Implementando o Método salvar()

```
public function salvar($obj) {  
    try {  
        $pdo = Conexao::conectar();  
  
        // Prepared statement para segurança  
        $stmt = $pdo->prepare(  
            "INSERT INTO alunos (nome, idade, email, curso)  
            VALUES (:nome, :idade, :email, :curso)"  
        );  
  
        // Vincula parâmetros para evitar SQL Injection  
        $stmt->bindParam(':nome', $obj->nome);  
        $stmt->bindParam(':idade', $obj->idade);  
        $stmt->bindParam(':email', $obj->email);  
        $stmt->bindParam(':curso', $obj->curso);  
  
        // Executa a inserção  
        return $stmt->execute();  
  
    } catch (PDOException $e) {  
        error_log("Erro ao salvar aluno: " . $e->getMessage());  
        return false;  
    }  
}
```

## 1. Prepare

Cria a consulta com placeholders

## 2. Bind

Vincula valores aos placeholders

## 3. Execute

Executa a operação com segurança

# Leitura de Dados com listar()

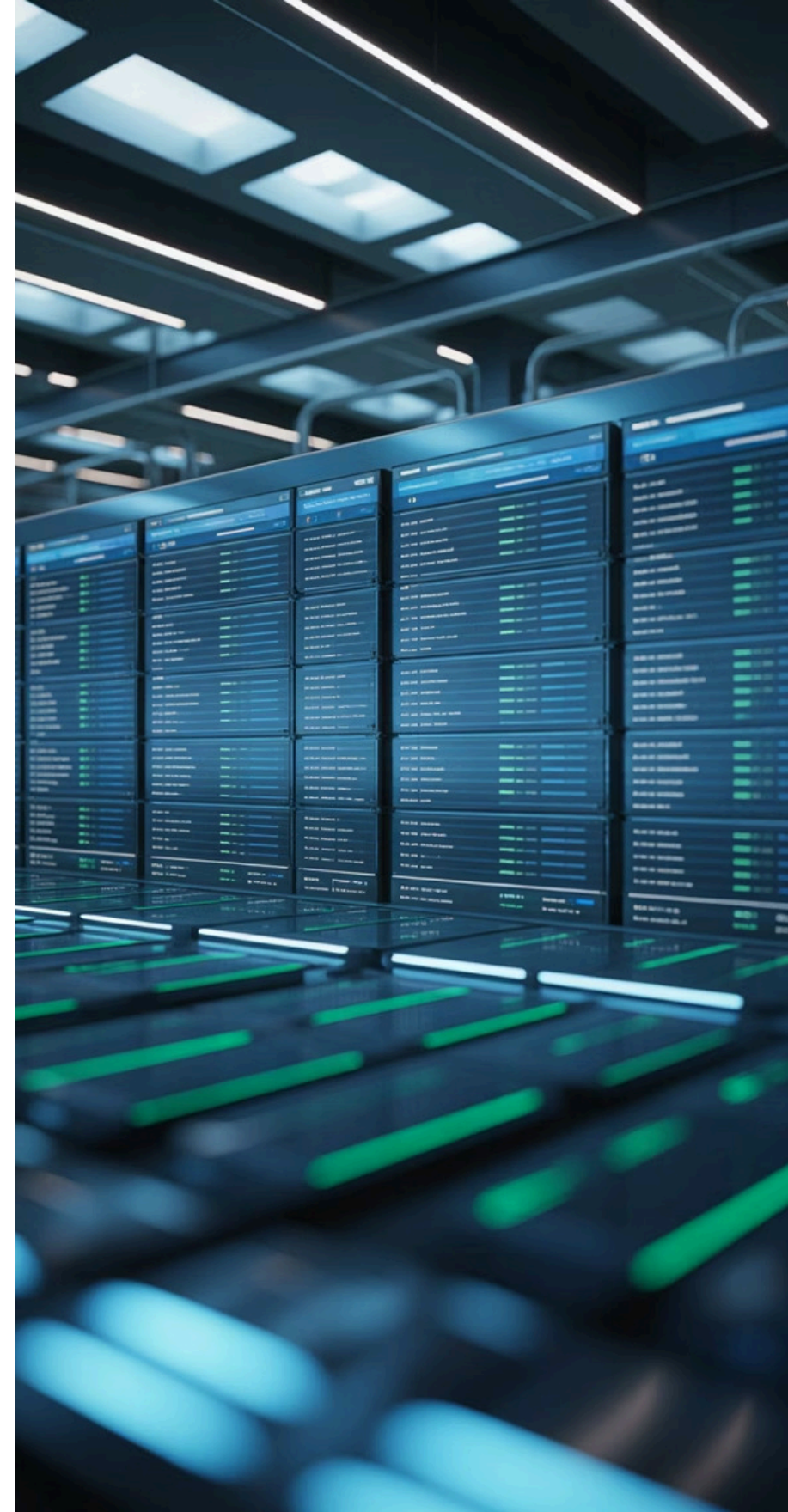
```
public function listar() {  
    try {  
        $pdo = Conexao::conectar();  
  
        // Consulta simples para buscar todos os registros  
        $stmt = $pdo->query(  
            "SELECT id, nome, idade, email, curso, data_cadastro  
            FROM alunos  
            ORDER BY nome ASC"  
        );  
  
        // Retorna array de objetos  
        return $stmt->fetchAll(PDO::FETCH_OBJ);  
  
    } catch (PDOException $e) {  
        error_log("Erro ao listar alunos: " . $e->getMessage());  
        return [];  
    }  
}
```

## Método fetchAll()

Retorna todos os registros de uma vez em formato de array. Ideal para listagens completas com poucos registros.

## PDO::FETCH\_OBJ

Converte cada linha em um objeto, permitindo acesso aos campos através de `$aluno->nome` em vez de `$aluno['nome']`.







# Fluxo Completo da Arquitetura

01

---

## Interface define o contrato

A interface `Repositorio` estabelece quais métodos devem existir (salvar, listar, atualizar, deletar)

02

---

## Classe abstrata fornece o modelo base

A classe `Modelo` implementa funcionalidades comuns e define métodos que devem ser customizados

03

---

## Classe concreta implementa a lógica

A classe `Aluno` cumpre o contrato da interface e estende o modelo com implementações específicas

04

---

## PDO realiza a persistência de dados

O PDO conecta com o banco de dados e executa operações CRUD de forma segura e abstrata

Este padrão arquitetural permite **escalabilidade**, **manutenibilidade** e **testabilidade** do código.



# Atividade Prática em Duplas

## CRUD Completo de Cadastro de Alunos

### Ferramentas



- PHP 7.4+
- PDO habilitado
- MySQL ou PostgreSQL
- Servidor local (XAMPP/WAMP)

### Componentes



- Interface Repositorio
- Classe abstrata Modelo
- Classe concreta Aluno
- Classe Conexao

### Operações



- **Create:** Cadastrar alunos
- **Read:** Listar e buscar
- **Update:** Editar dados
- **Delete:** Remover registros

 **Importante:** Interface visual HTML é opcional, mas recomendada para melhor visualização dos resultados.



# Passo 1: Criar Banco de Dados

## Estrutura da Base escola e Tabela alunos

```
-- Criar banco de dados
CREATE DATABASE IF NOT EXISTS escola
CHARACTER SET utf8mb4
COLLATE utf8mb4_unicode_ci;

USE escola;

-- Criar tabela de alunos
CREATE TABLE alunos (
  id INT AUTO_INCREMENT PRIMARY KEY,
  nome VARCHAR(100) NOT NULL,
  idade INT NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  curso VARCHAR(50) NOT NULL,
  data_cadastro TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
  data_atualizacao TIMESTAMP DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  INDEX idx_nome (nome),
  INDEX idx_email (email)
) ENGINE=InnoDB DEFAULT CHARSET=utf8mb4;
```

### Campos Essenciais

- id: Chave primária auto-incremento
- nome, idade, email, curso: Dados do aluno
- Timestamps para auditoria

### Boas Práticas

- Charset UTF-8 para acentuação
- Índices para busca rápida
- Constraints de integridade

# Passo 2: Classes Base (Conexão, Interface e Modelo)

## Implementando a Fundação do Sistema

### Conexao.php

```
class Conexao {  
    private static $pdo = null;  
  
    public static function conectar() {  
        if (!self::$pdo) {  
            self::$pdo = new PDO(  
                'mysql:host=localhost;  
                dbname=escola',  
                'root', ''  
            );  
        }  
        return self::$pdo;  
    }  
}
```

### Repositorio.php

```
interface Repositorio {  
    public function salvar($obj);  
    public function listar();  
    public function buscarPorId($id);  
    public function atualizar($obj);  
    public function deletar($id);  
}
```

### Modelo.php

```
abstract class Modelo {  
    protected $id;  
  
    public function getId() { return $this->id; }  
    public function setId($id) { $this->id = $id; }  
  
    abstract public function validar();  
}
```



# Passo 3: Classe Aluno com CRUD Completo

```
class Aluno extends Modelo implements Repositorio {
    private $nome;
    private $idade;
    private $email;
    private $curso;

    // Implementação do método abstrato
    public function validar() {
        if (empty($this->nome) || empty($this->email)) {
            throw new Exception("Nome e email são obrigatórios");
        }
        if ($this->idade < 16 || $this->idade > 100) {
            throw new Exception("Idade deve estar entre 16 e 100 anos");
        }
        if (!filter_var($this->email, FILTER_VALIDATE_EMAIL)) {
            throw new Exception("Email inválido");
        }
        return true;
    }

    // Implementação dos métodos da interface Repositorio
    public function salvar($obj) { /* código do slide 10 */ }
    public function listar() { /* código do slide 11 */ }
    public function buscarPorId($id) { /* implementar */ }
    public function atualizar($obj) { /* implementar */ }
    public function deletar($id) { /* implementar */ }

    // Getters e Setters...
}
```

A classe **Aluno** agora possui toda a estrutura necessária para realizar operações CRUD completas de forma segura e validada.



# Passo 4: Páginas PHP para Interface

## form\_aluno.php

Formulário para **cadastro** e **edição** de alunos

- Campos: nome, idade, email, curso
- Validação client-side e server-side
- Modo de criação ou edição (por ID)
- Feedback de sucesso/erro



## listar\_alunos.php

Listagem de alunos com opções de **edição** e **exclusão**

- Tabela HTML com todos os alunos
- Botão "Editar" (redireciona ao form)
- Botão "Excluir" (com confirmação)
- Link para cadastrar novo aluno



 **Dica:** Use Bootstrap ou CSS simples para melhorar a aparência das páginas e tornar a interface mais amigável.



# Passo 5: Testar Operações e Validar

## Checklist de Testes

1

### CREATE – Cadastrar Aluno

Preencha o formulário com dados válidos e verifique se o registro é criado no banco. Teste também validações (email inválido, idade fora do range).

2

### READ – Listar Alunos

Acesse a página de listagem e confirme que todos os alunos cadastrados aparecem corretamente ordenados por nome.

3

### UPDATE – Editar Aluno

Clique em "Editar", modifique os dados e salve. Verifique se as alterações foram persistidas no banco de dados.

4

### DELETE – Excluir Aluno

Clique em "Excluir", confirme a ação e verifique se o registro foi removido tanto da interface quanto do banco.

Teste cenários de **erro** (duplicação de email, campos vazios) e **sucesso** para garantir robustez.





# Discussão em Grupo

## Como reuso e abstração ajudam na manutenção?

- Redução de código duplicado
- Alterações centralizadas propagam automaticamente
- Facilita identificação e correção de bugs
- Novos recursos aproveitam estrutura existente
- Testes mais simples e focados

## Diferenças entre MySQL e PostgreSQL no PDO?

- **DSN:** mysql:host vs pgsql:host
- **Auto-increment:** AUTO\_INCREMENT vs SERIAL
- **Tipos de dados:** diferenças em DATETIME, BOOLEAN
- **Funções:** NOW() vs CURRENT\_TIMESTAMP
- **PDO abstrai:** mesmo código PHP para ambos!

"O uso de interfaces e abstração transforma código difícil de manter em arquitetura escalável e profissional."